

SISTEMAS DE CONTROL I

Trabajo final integrador

Sistema de control de temperatura para un CPU

Docentes:

AGUERO CLAUDIO (Titular)
PEDRONI JUAN PABLO (Adjunto)

Alumnos:

Bernaus Julieta, Campos Mariano

10 de febrero de 2025



Resumen

Este proyecto tiene como objetivo diseñar e implementar un sistema de control a lazo cerrado para regular la temperatura de un CPU mediante el ajuste dinámico de la velocidad de un ventilador. El sistema busca mantener la temperatura dentro de límites seguros, optimizando el desempeño y la vida útil del CPU.

Índice

1. Definición del problema	3
1.1. Principio de funcionamiento del sistema	3
1.2. Variable a controlar	3
1.3. Medición de la variable de salida	3
1.4. Ejecución de la acción de control	3
1.5. Variables del sistema	3
1.6. Posibles perturbaciones	4
1.7. No linealidades involucradas	4
1.8. Niveles de señal de entrada y salida	4
2. Análisis de la planta	5
2.1. Diagrama de bloques del sistema	6
2.2. Modelado matemático de los componentes	7
2.2.1. Sensor de temperatura	7
2.2.2. Ventilador	7
2.2.3. Comparador	9
2.3. Función de transferencia global	12
2.4. Análisis de estabilidad absoluta y relativa	13
2.5. Análisis de la respuesta temporal	14
2.5.1. Determinación del tipo de sistema	16
2.5.2. Polos dominantes	16
3. Especificaciones de diseño	17
4. Diseño del controlador	18
5. Simulación	24
6. Conclusiones	26
7. Bibliografía	27

1. Definición del problema

1.1. Principio de funcionamiento del sistema

La manera en la que los CPU de las computadoras modernas regulan su temperatura es a través de sensores de temperatura que dan la orden a diferentes sistemas de enfriamiento para mantener la temperatura en un rango deseado. Una de las opciones más populares para enfriar es un ventilador. Mantener las temperaturas bajas es crucial para garantizar el correcto funcionamiento de una computadora, ya que si no, puede llevar a que la computadora se apague al alcanzar los límites superiores para prevenir daños permanentes a sus componentes. Basado en este concepto, el sistema propuesto es un controlador a lazo cerrado para regular la temperatura de un CPU mediante el ajuste de la velocidad de un ventilador. El principio de funcionamiento se basa en la retroalimentación: se mide constantemente la temperatura del CPU, se compara con un valor deseado (setpoint), y se ajusta dinámicamente la velocidad del ventilador para mantener la temperatura en los límites establecidos.

1.2. Variable a controlar

La variable a controlar es la temperatura del CPU, (T_{CPU} en grados Celsius, °C). El objetivo es mantenerla dentro de un rango seguro para evitar el sobrecalentamiento, con un setpoint ajustable dependiendo de la carga del sistema.

1.3. Medición de la variable de salida

Si bien algunos CPU tienen incorporados sensores de temperaturas internos, para nuestro caso utilizamos un sensor de temperatura LM35.

- Sensor: precisión de $\pm 0.5^\circ\text{C}$.
- Acondicionamiento de señal: el LM35 podría requerir un ADC si fuera necesario utiliza un microcontrolador (Se determina en las secciones posteriores).

1.4. Ejecución de la acción de control

La acción de control se ejecutará mediante un ventilador de corriente continua (DC) cuyo motor será controlado con señales PWM (modulación por ancho de pulso). La señal PWM ajustará las revoluciones por minuto (RPM) del ventilador, proporcionalmente a la señal de control generada por el comparador.

1.5. Variables del sistema

Variable	Unidad	Descripción
T_{CPU}	°C	Temperatura del CPU.
V_{Fan}	RPM	Velocidad del ventilador.
Señal de control (u)	% (Duty Cycle)	Señal generada por el controlador (PWM).
T_{Amb}	°C	Temperatura ambiente (perturbación).

Cuadro 1: Variables del sistema con sus unidades y descripciones.

1.6. Posibles perturbaciones

- Variaciones en la temperatura ambiente T_{Amb} : el sistema debe compensar los cambios en la temperatura externa.
- Carga del CPU: a mayor carga, se genera más calor, lo que afecta directamente la variable controlada, la T_{CPU} .
- Fluctuaciones de voltaje en el suministro eléctrico: pueden alterar el funcionamiento del ventilador. No se tiene en cuenta para el diseño.

1.7. No linealidades involucradas

El sistema de control que regula la temperatura del CPU a través del ventilador debe abordar múltiples fuentes de no linealidad:

- Ventilador: la relación entre el ciclo de trabajo PWM y la velocidad del ventilador es no lineal, especialmente a bajas RPM.
- Sensores de temperatura: los sensores tienen respuestas no lineales que deben ser compensadas para obtener mediciones precisas.
- Transferencia de calor: el comportamiento térmico del CPU y su interacción con el sistema de enfriamiento (ventilador) es no lineal.

1.8. Niveles de señal de entrada y salida

Existen dos señales principales, donde es de especial interés conocer sus rangos dinámicos: la T_{CPU} y por otro lado la señal de control PWM . Respecto a la primera señal, tenemos que la temperatura máxima que puede alcanzar un CPU depende de varios factores, como el modelo específico del procesador, el sistema de refrigeración utilizado, y las condiciones ambientales. Se debe tener en cuenta que:

- Temperatura nominal de operación: un CPU típico a máxima carga generalmente puede alcanzar temperaturas entre 70°C y 90°C. Este rango depende del tipo de CPU (por ejemplo, Intel o AMD), el proceso de fabricación, y las condiciones de refrigeración.
- Temperatura crítica o límite superior: los fabricantes de CPUs suelen establecer una temperatura máxima segura alrededor de los 100°C a 105°C. Si el CPU alcanza estas temperaturas, el sistema activará medidas de protección, como el thermal throttling (reducción de la velocidad de reloj) o el apagado del sistema para evitar daños.

De lo mencionado anteriormente se concluye que la T_{CPU} oscila entre 30°C y 100°C, y por tanto teniendo en cuenta el datasheet del sensor LM35, el rango dinámico de la señal resulta de 300[mV] hasta 1[V]. Para la segunda señal de interés tenemos la señal PWM, esta varía según el ciclo de trabajo desde 0 % hasta el 100 %, esto se traduce en un rango dinámico de 0[V] hasta 5[V](5000[RPM]).

2. Análisis de la planta

En este caso, la planta es el CPU con su disipador. Para obtener el modelo matemático, se recurre a la ley de Ohm térmica: la inercia térmica de los cuerpos (CPU y disipador) se modela como una capacidad, se tiene:

$$I[A] = \frac{\Delta V[V]}{R[ohm]} = Q[W] = \frac{\Delta T[C]}{Rth[C/W]} \quad (1)$$

$$Cth[J/C] \quad (2)$$

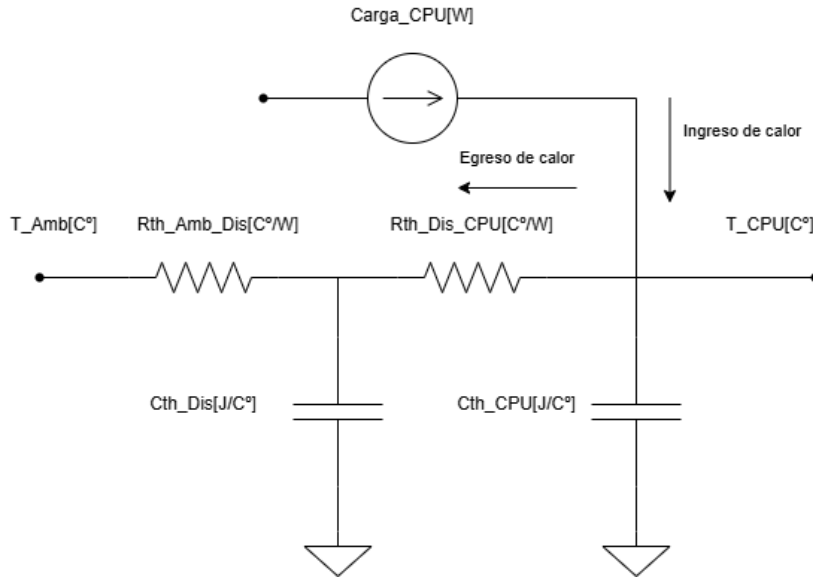


Figura 1: Modelo de la planta

Para la perturbación producida por la carga del CPU, esta aporta una cantidad de calor Q a la planta, por lo que se puede modelar con una fuente de corriente. Como resultado del modelo, tenemos dos flujos de calor: el entrante por la carga del CPU y el saliente debido a que $T_{Amb} < T_{CPU}$. Se calcula la función de transferencia con el siguiente código de Matlab:

```

1  % Declarar variables simbolicas
2  syms T_Amb T_CPU T_Dis
3  syms Rth_Amb_Dis Rth_Dis_CPU Cth_Dis Cth_CPU
4  syms s
5
6  % Temperaturas y flujo de calor
7  % Primera etapa: T_Amb -> T_Dis
8  Q1 = (T_Amb - T_Dis) / Rth_Amb_Dis; % Flujo de calor a traves de
9  eq1 = Q1 == Cth_Dis * s * T_Dis;    % Relacion por la capacidad
10                                     % termica
11
12 % Segunda etapa: T_Dis -> T_CPU
13 Q2 = (T_Dis - T_CPU) / Rth_Dis_CPU; % Flujo de calor a traves de
14                                     % Rth_Dis_CPU
15 eq2 = Q2 == Cth_CPU * s * T_CPU;    % Relacion por la capacidad
                                     % termica
16
17 % Resolver el sistema de ecuaciones

```

```

16 % Se despeja T_Dis y T_CPU en terminos de T_Amb
17 sol = solve([eq1, eq2], [T_Dis, T_CPU]);
18
19 % Obtener T_CPU como funcion de T_Amb
20 T_CPU_T_Amb = collect(simplify(sol.T_CPU / T_Amb), s)
21
22 Rth_Amb_Dis = 0.5; % Resistencia termica entre T_Amb y T_Dis [C/W]
23 Rth_Dis_CPU = 0.3; % Resistencia termica entre T_Dis y T_CPU [C/W]
24 Cth_Dis = 10; % Capacitancia termica en el nodo T_Dis [J/C]
25 Cth_CPU = 5; % Capacitancia termica en el nodo T_CPU [J/C]
26
27 T_CPU_T_Amb=eval(T_CPU_T_Amb)

```

Como resultado del análisis de obtenemos:

$$FdT_{Planta} = \frac{T_{CPU}(s)}{T_{Amb}(s)} = \frac{1}{7,5s^2 + 6,5s + 1} \quad (3)$$

2.1. Diagrama de bloques del sistema

Se plantea un diagrama de bloques para modelar el sistema en su totalidad, se identifican sus partes principales(comparador, planta, sensor), las variables de entrada, salida y perturbaciones, ademas se detalla como interactúan estas entre si.

- Entrada (señal de referencia): la temperatura deseada o setpoint (se puede suponer una temperatura objetivo o de referencia a mantener para el CPU, T_{ref}).
- Comparador: compara la temperatura medida con la de referencia y genera una señal de control en forma de PWM para el ventilador.
- Actuador: recibe la señal PWM del comparador y ajusta su velocidad.
- Perturbaciones: la carga del CPU es la principal perturbación que afecta la variable de salida.
- Sensor: mide la temperatura real del CPU, T_{CPU} y esta señal es retroalimentada al comparador.
- Planta: el CPU es el elemento a controlar, las variables de entrada son las perturbaciones que afectan su temperatura, y la refrigeración que aporta el ventilador, la salida es el processvalue T_{CPU}

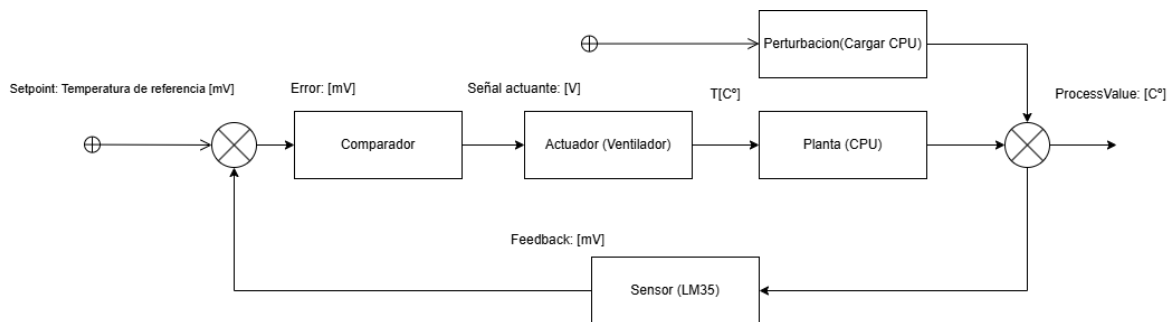


Figura 2: Sistema completo

2.2. Modelado matemático de los componentes

2.2.1. Sensor de temperatura

En el caso de la función de transferencia del sensor de temperatura LM35, esta se puede obtener de forma teórica, ya que en el datasheet se encuentra la expresión $V_{OUT} = F(T_{Amb})$, para la aplicación típica (FIGURE 1. Basic Centigrade Temperature Sensor), la expresión resulta:

$$V_{OUT}[V] = 0[V] + 0,01[V].T_{Amb}[C] \quad (4)$$

La función de transferencia de interés, se obtienen pasando la ecuación al dominio operacional y despejando la expresión *Voltaje/Temperatura*, esta resulta:

$$FdT_{Sensor} = \frac{V_{LM35}(s)}{T_{CPU}(s)} = 0,01 \quad (5)$$

2.2.2. Ventilador

Para obtener la función de transferencia del ventilador se optó por hacerlo de forma experimental, ya que la relación entrada-salida de la señal PWM y temperatura, no se encontraba disponible en ninguna bibliografía. El procedimiento es el siguiente:

- Armar el sistema de refrigeración con el ventilador y el habitáculo donde se va a encontrar el CPU.
- Hacer variar la señal de entrada PWM del ventilador y medir la temperatura resultante en el habitáculo.
- Con los datos obtenidos, mediante un script en Matlab, obtener la función de transferencia

Nota: En nuestro caso, los datos son simulados con un script de Python.

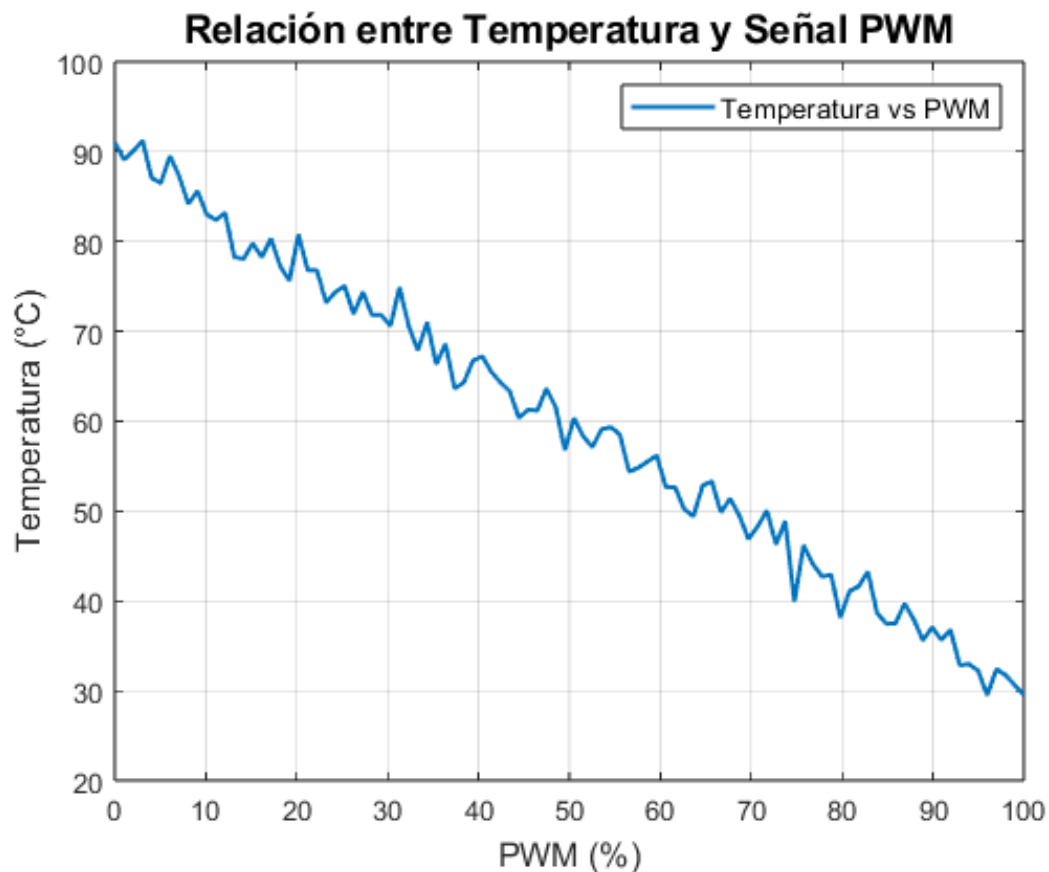


Figura 3: Datos recopilados

Con los datos obtenidos, se realiza el siguiente análisis para obtener la función de transferencia:

```

1  %Calculo de funcion de transferencias para el ventilador
2  %Importamos los datos obtenidos de las mediciones
3  opts = delimitedTextImportOptions(" NumVariables", 2);
4  opts.DataLines = [2, Inf];
5  opts.Delimiter = ",";
6  opts.VariableNames = ["PWM", "TemperatureC"];
7  opts.VariableTypes = ["double", "double"];
8  opts.ExtraColumnsRule = "ignore";
9  opts.EmptyLineRule = "read";
10
11 % Leer el archivo CSV como tabla
12 data = readtable("D:\GD\Sistemas de control I\Trabajo-Integrador-
    SCI\Datos\pwm_temperature_data.csv", opts);
13
14 % Separar las columnas en variables
15 PWM = data("PWM"); % Columna PWM
16 Temperature = data("TemperatureC"); % Columna Temperatura
17
18 % Crear el grafico
19 figure;
20 plot(PWM, Temperature, 'LineWidth', 1.5, 'MarkerSize', 6, '
    DisplayName', 'Temperatura vs PWM');
21 grid on;
22 xlabel('PWM (%)', 'FontSize', 12);
23 ylabel('Temperatura (C)', 'FontSize', 12);
24 title('Relacion entre Temperatura y Senal PWM', 'FontSize', 14);

```



```

25 legend('show', 'Location', 'northeast', 'FontSize', 10);
26
27
28 % Crear entrada y salida como senales en tiempo (asumiendo un
    muestreo uniforme)
29 t = linspace(0, length(PWM)-1, length(PWM)); % Tiempo ficticio
30 u = PWM; %Entrada (PWM)
31 y = Temperature; % Salida (Temperatura)
32
33 % Ajustar un modelo de primer o segundo orden (dominio Laplace)
34 data_id = iddata(y, u, 1); % Crear datos de identificacion con un
    muestreo ficticio de 1s
35 model = tfest(data_id, 1, 0); % Ajustar un modelo de primer orden
    sin ceros
36
37 % Mostrar la funcion de transferencia estimada
38 disp('Funcion de transferencia estimada:');
39 disp(model);
40
41 % Graficar comparacion entre el modelo ajustado y los datos
42 figure;
43 compare(data_id, model);
44 title('Comparacion entre datos reales y modelo ajustado', 'FontSize', 14);
45 xlabel('Tiempo (s)', 'FontSize', 12);
46 ylabel('Temperatura (C)', 'FontSize', 12);
47 legend('Datos reales', 'Modelo ajustado', 'Location', 'best', 'FontSize', 10);
48 grid on;

```

La función de transferencia obtenida resulta:

$$FdT_{Ventilador} = \frac{T_{Amb}(s)}{PWM(s)} = \frac{-0,0038}{s + 0,007} \quad (6)$$

2.2.3. Comparador

Para obtener la función de transferencia del comparador, suponemos una $V_{ref} = 600[mV]$. Esta se condice con una temperatura de $60^{\circ}C$. La señal de entrada va ser el error que resulta de la resta $V_{ref} - V_{LM35}$; en el caso de que el error sea $0[V]$, la salida de PWM al ventilador es cero. Esto significa que está operando a una temperatura segura. Para el caso donde el error sea $-300[mV]$, es decir que la temperatura del CPU ronda los $90^{\circ}C$, la salida PWM debe ser $5[V]$. Entre medio de estos dos valores definimos una variación lineal; para errores positivos la salida de PWM también debe ser $0[V]$ esto significa que esta por debajo de los $60^{\circ}C$. Con este análisis, calculamos una recta que pase por estos dos puntos, para luego calcular la función de transferencia.

Comparación entre datos reales y modelo ajustado

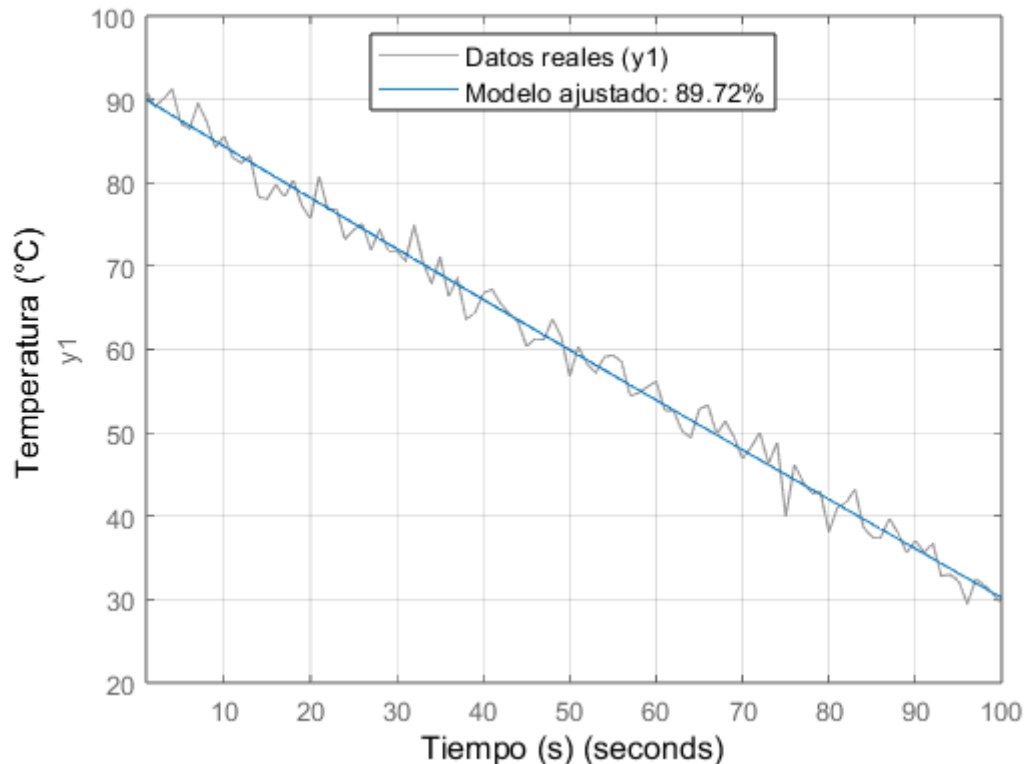


Figura 4: Ajuste del modelo

```

1  % Parametros iniciales
2  V_ref = 0.6; % [V] Referencia de temperatura (60 C)
3  V_LM35_safe = V_ref; % Valor seguro donde el error es 0 [V]
4  V_LM35_danger = 0.3; % [V] (90 C -> 600 - 300 = -0.3 [V])
5
6  % Valores de PWM en los puntos limite
7  PWM_safe = 0; % PWM = 0 [V] (Temperatura segura)
8  PWM_danger = 5; % PWM = 5 [V] (Temperatura alta)
9
10 % Calculo de la pendiente de la recta
11 % Recta entre los puntos: (-0.3, 5) y (0, 0)
12 m = (PWM_danger - PWM_safe) / (V_LM35_safe - V_LM35_danger);
13
14 % Ordenada al origen de la recta
15 b = PWM_safe - m * (V_LM35_safe - V_ref);
16
17 % Mostramos la funcion lineal calculada
18 fprintf('La funcion PWM respecto al error (V_ref - V_LM35) es:\n');
19 fprintf('PWM = %.2f * error + %.2f\n', m, b);
20
21 % Crear el modelo de funcion de transferencia
22 % Supongamos entrada U(s) (error) y salida Y(s) (PWM)
23 % Relacion directa en dominio del tiempo: PWM = m * error + b
24 numerator = [m]; % Numerador (ganancia m)
25 denominator = [1]; % Denominador (sistema de primer orden, estatico)
26 sys = tf(numerator, denominator);
27
28 % Visualizar la respuesta del sistema

```

```

29 % Definir un rango de errores desde -300 mV a +300 mV
30 error = linspace(-0.3, 0.3, 100); % [V]
31 PWM = m * error + b;
32
33 % Forzar PWM a ser cero para errores positivos
34 PWM(error > 0) = 0;
35
36 % Graficar
37 figure;
38 plot(error * 1000, PWM);
39 grid on;
40 xlabel('Error [mV]');
41 ylabel('Salida PWM [V]');
42 title('Relacion entre Error y PWM');
43 legend('PWM vs Error');

```

La ecuación de la recta es:

$$PWM[V] = 16,67Error[V] - 0 \quad (7)$$

La función de transferencia es simplemente la pendiente. Esto representa una ganancia constante entre el error y la señal PWM de salida. Notar que la función de transferencia solo captura las dependencias entre la salida y la entrada. Los términos independientes (constantes) no afectan la relación entre la entrada y la salida en términos de función de transferencia. El resultado del análisis anterior es:

$$FdT_{Comparador} = \frac{PWM(s)}{Error(s)} = 16,67 \quad (8)$$

2.3. Función de transferencia global

Con todos los elementos del sistema modelados, calculamos la función de transferencia global. La obtención de esta se realiza mediante el álgebra de bloques con el siguiente script de matlab:

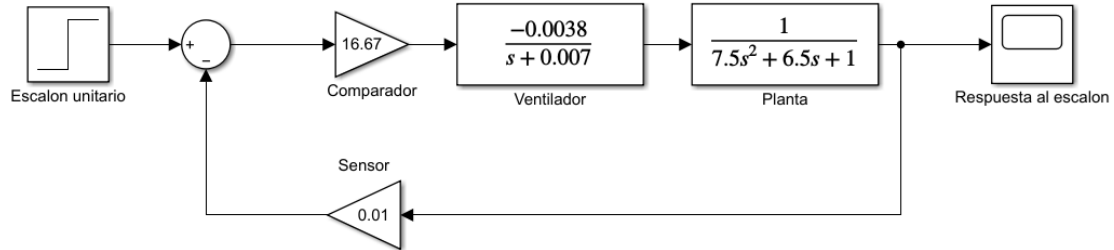


Figura 5: Diagrama de bloques del sistema

```

1  % Definir la variable simbolica para la funcion de transferencia
2  s = tf('s');
3
4  % Funciones de transferencia individuales
5  FdT_Planta = 1/(7.5*s^2 + 6.5*s + 1);           % Funcion de
6  % transferencia de la planta
7  FdT_Sensor = 0.01;                             % Funcion de
8  % transferencia del sensor
9  FdT_Ventilador = -0.0038/(s + 0.007);           % Funcion de
10 % transferencia del ventilador
11 FdT_Comparador = 16.67;                         % Funcion de
12 % transferencia del comparador
13
14 %Utilizando el algebra de bloques obtenemos la fdt global
15 G=FdT_Planta*FdT_Ventilador*FdT_Comparador;
16 H=FdT_Sensor;
17
18 %Funcion de transferencia global
19 FdT_Global=feedback(G,H);
20
21 % Funcion de transferencia global original
22 disp('La funcion de transferencia global original es:');
23 FdT_Global

```

La función de transferencia global resulta:

$$FdT_{Global} = \frac{-0,06335}{7,5s^3 + 6,553s^2 + 1,046s + 0,006367} \quad (9)$$

2.4. Análisis de estabilidad absoluta y relativa

Para probar la estabilidad absoluta de una función de transferencia, se puede analizar los polos de la misma. Un sistema es estable de forma absoluta si todos sus polos tienen partes reales negativas. Esto implica que, para que el sistema sea estable, la solución a la ecuación característica asociada a la función de transferencia debe tener raíces con partes reales negativas. Por otro lado, valuamos la estabilidad relativa de sistema con herramientas gráficas como el lugar de raíces, que muestra cómo evolucionan las posiciones de los polos a lazo cerrado cuando varía el valor de k_p .

```
1 % Definir la funcion de transferencia a lazo abireto (GH)
2 GH = FdT_Comparador*FdT_Ventilador*FdT_Planta*FdT_Sensor
3
4 % Calcular y mostrar los ceros de la funcion de transferencia a lazo
5 % abierto
6 disp('Ceros de la funcion de transferencia:');
7 zero_GH = zero(GH);
8 disp(zero_GH);
9
10 % Calcular y mostrar los polos de la funcion de transferencia a lazo
11 % abierto
12 disp('Polos de la funcion de transferencia:');
13 pole_GH = pole(GH);
14 disp(pole_GH);
15
16 % Graficar el lugar de raices de la funcion de transferencia
17 figure;
18 rlocus(GH);
19 grid on;
20
21 %El sistema es marginalmente estable , para k<11
22 rlocfind(GH);
23 grid on;
24
25 title('Lugar de Raices de la Funcion de Transferencia');
26 xlabel('Parte Real');
27 ylabel('Parte Imaginaria');
```

Las raíces de la ecuación característica son:

$$P1 = -0,6667, P2 = -0,2000, P3 = -0,0070 \quad (10)$$

Todas las raíces de la ecuación característica tienen parte real negativa, podría decirse que el sistema es estable de forma absoluta, de forma relativa podemos decir que para valores de k menores a 11 el sistema es estable y para valores mayores el deja de serlo.

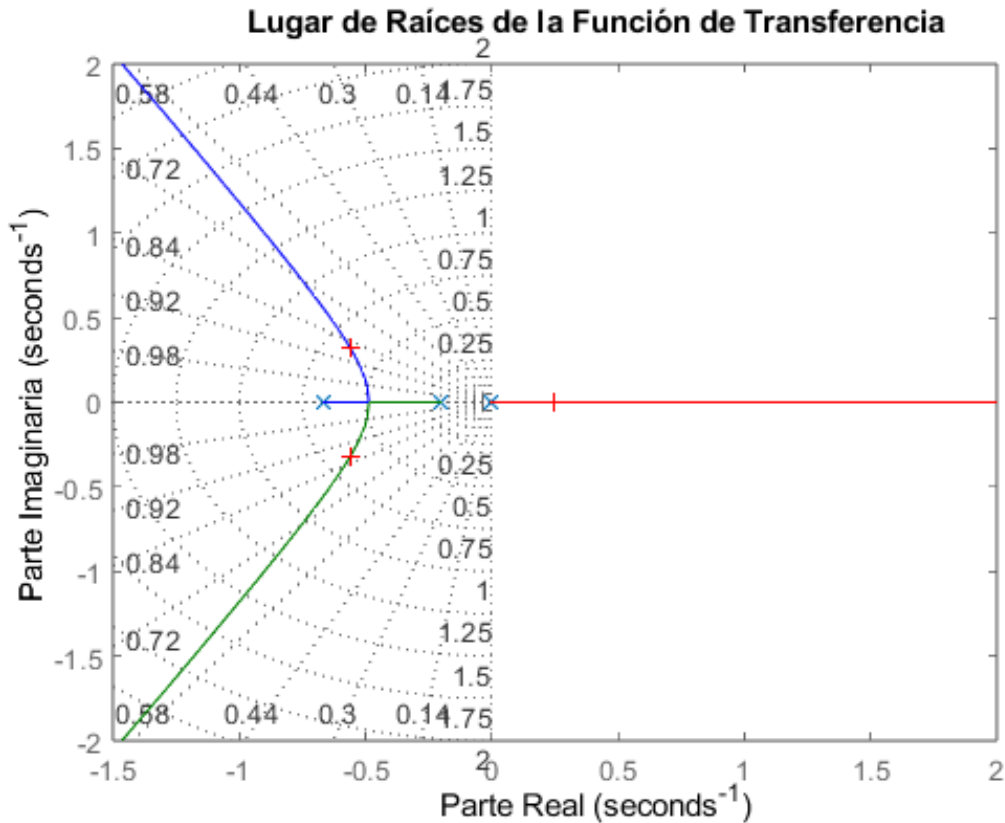


Figura 6: Lugar de raíces de FdTGlobal

2.5. Análisis de la respuesta temporal

Verificamos el comportamiento del sistema simulando la respuesta al escalón unitario con distintos valores de k_p . La forma de respuesta es decreciente: esto se debe a que aumentar la señal de entrada disminuye la salida (ganancia negativa). Este comportamiento es esperable, ya que la temperatura debe disminuir cuando la señal de control (PWM) aumenta.

```

1  %Definimos distintos valores de k para simular la respuesta
2  k1 = 5;
3  k2 = 20;
4
5  %Por la ubicacion del polo dominante resulta imposible una
6  %solo proporcional ya que el polo dominante esta muy cerca del origen
7  %y
8  %hace al sistema muy lento
9
10 % Crear un figure con tres subplots
11 figure;
12
13 % Primer subplot: Respuesta para k1
14 subplot(2, 1, 1);
15 step(feedback(k1*G,H));
16 title('kp1=5 "Estable"');
17 xlabel('Time')
18 ylabel('Amplitude')
19 grid on;

```

```

20 % Segundo subplot: Respuesta para k2
21 subplot(2, 1, 2);
22 step(feedback(k2*G,H));
23 title('kp2=20 "Inestable"');
24 xlabel('Time')
25 ylabel('Amplitude')
26 grid on;
27
28 figure;
29 step(feedback(G,H));
30 title('Respuesta temporal sin kp');
31 xlabel('Time')
32 ylabel('Amplitude')
33 grid on

```

En el resultado de la simulación, podemos ver que el sistema es estable para $kp = 5$ e inestable para $kp = 20$. Este resultado es esperable de acuerdo al análisis de estabilidad relativa que se realizó anteriormente.

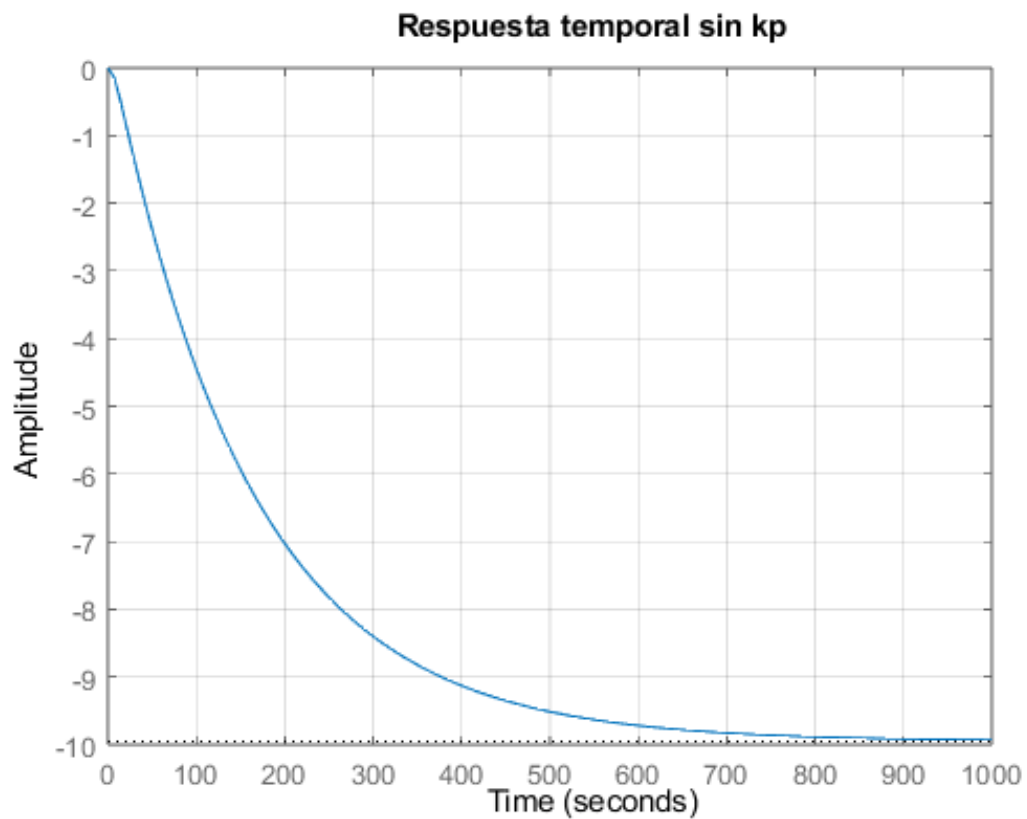


Figura 7: Respuesta temporal sin compensación proporcional

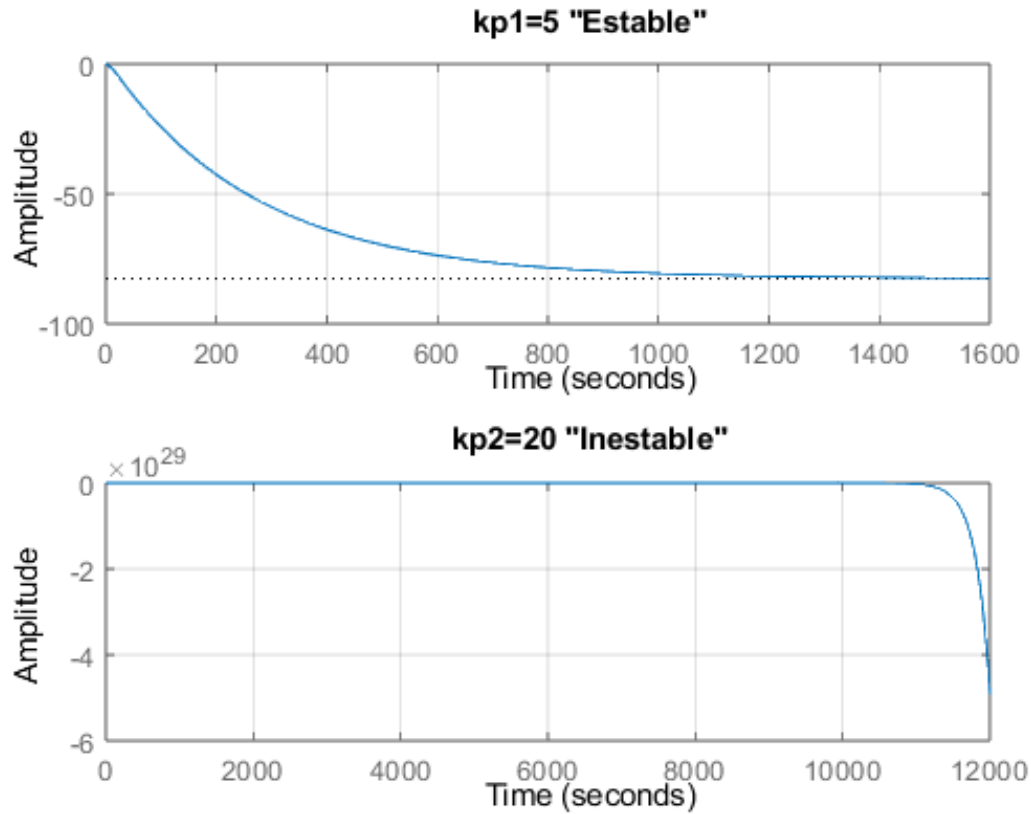


Figura 8: Respuesta al escalón unitario

2.5.1. Determinación del tipo de sistema

Para determinar el tipo de sistema, nos enfocamos en la cantidad de integradores en la trayectoria directa, lo cual se observa en la cantidad de polos en el origen ($s = 0$) de la función de transferencia a lazo abierto. En este caso, la función de transferencia no tiene ningún factor 's' aislado en el denominador. Por lo tanto, este sistema es de Tipo 0. Esto quiere decir que el error de estado estable para una entrada escalón es constante; en nuestro caso, esta representa a la temperatura de referencia (setpoint), por lo que resulta de interés tener el menor error de estado estable posible.

2.5.2. Polos dominantes

Respecto a los polos dominantes, en este caso lo tenemos en $p3 = -0,007$, su proximidad al origen hace que la respuesta del sistema sea demasiado lenta (esto es esperable en sistemas térmicos). Podemos ver en la respuesta temporal que el tiempo de establecimiento está en el orden de los 700 segundos; este valor dista mucho de las especificaciones de diseño por lo que resulta imperioso corregirlo al momento de compensar el sistema.

3. Especificaciones de diseño

Partimos de las especificaciones de diseño en el dominio del tiempo, los requerimientos son:

$$ess = \min \quad (11)$$

$$ts \leq 50[s] \quad (12)$$

$$Mp \leq 4\% \quad (13)$$

Para un primer análisis, podríamos tratar de implementar un compensador PID que cancele los dos polos dominantes $p2 = -0,2$ y $p3 = -0,007$, esto permite mejorar la respuesta temporal (hacerlo mas rápido al sistema) y al mismo tiempo mejorar el error de estado estable (aumenta el tipo). Ambas cosas son deseables, sin embargo el hecho de que el sistema tenga una ganancia negativa y un polo al origen hace que el sistema compensado sea inestable para todo k_p , como se muestra en el siguiente análisis.

4. Diseño del controlador

Análisis para la implementación de para el compensador:

```
1  % Definir la variable simbolica para la funcion de transferencia
2  s = tf('s');
3
4  % Funciones de transferencia individuales
5  FdT_Planta = 1/(7.5*s^2 + 6.5*s + 1);           % Funcion de
6  % transferencia de la planta
7  FdT_Sensor = 0.01;                             % Funcion de
8  % transferencia del sensor
9  FdT_Ventilador = -0.0038/(s + 0.007);          % Funcion de
10 % transferencia del ventilador
11 FdT_Comparador = 16.67;                         % Funcion de
12 % transferencia del comparador
13
14 % Definir la funcion de transferencia a lazo abierto (GH)
15 G=FdT_Planta*FdT_Ventilador*FdT_Comparador;
16 H=FdT_Sensor;
17 disp('La funcion de transferencia a lazo abierto "GH" ');
18 GH=G*H
19
20 % Mostrar la funcion de transferencia a lazo abierto en forma de
21 % ceros y polos
22 disp('La funcion de transferencia a lazo abierto explicita');
23 GH=zpk(GH)
24
25 % La ecuacion caracteristica de la funcion de transferencia a lazo
26 % abierto resulta ser:
27 %  $E_c(s) = (s+0.6667)(s^2 + 0.207s + 0.0014)$ 
```

Para cancelación de polos dominantes con PID tenemos:

```

1  % La forma del PID es: PID = kp * (Ti*Td*s^2 + Ti*s + 1) / (Ti*s)
2
3  % Vamos a igualar terminos con la ecuacion de segundo orden en el
4  % denominador
5
6  % Comparacion con la ecuacion de segundo orden:
7  % a^2 + b*s + c = s^2 + 0.207*s + 0.0014
8
9  c = 1; % Valor de c (termino constante)
10 b = 0.207/0.0014; % Calculamos b
11 a = 1/0.0014; % Calculamos a
12
13 Ti = b
14 Td = a/Ti
15
16 % Armamos el controlador PID utilizando los valores de Ti y Td
17 PID =(Ti*Td*s^2 +Ti*s+1)/(Ti*s)
18
19 % Armamos la funcion de transferencia a lazo abierto compensado (con
20 % PID)
21 FdT_Global_PID = PID*GH
22
23 % Verificamos la cancelacion de polos
24 figure;
25 subplot(2,1,1); % Primer subplot para la funcion a
26 % lazo abierto
27 pzmap(GH); % Muestra el mapa de polos y ceros de la funcion a
28 % lazo abierto
29 title('Mapa de Polos y Ceros a lazo abierto "GH"');
30 xlabel('real');
31 ylabel('imaginaria');
32 grid on;
33
34 subplot(2,1,2); % Segundo subplot para la funcion con PID
35 pzmap(FdT_Global_PID); % Muestra el mapa de polos y ceros de la
36 % funcion compensada
37 title('Mapa de Polos y Ceros con Controlador PID');
38 xlabel('real');
39 ylabel('imaginaria');
40 grid on;
41
42 % Simplificamos la expresion para obtener una forma mas compacta
43 FdT_Global_PID = minreal(FdT_Global_PID) % Simplificacion de la fdt
44 figure;
45 rlocus(FdT_Global_PID);
46 grid on;
47 title('Lugar de raices de la funcion de transferencia compensada');

```

La función de transferencia del controlador PID resulta:

$$PID(s) = \frac{714,3s^2 + 147,9s + 1}{147,9s} \quad (14)$$

La función a lazo cerrado global compensada es:

$$FdT_{GlobalComp} = \frac{0,4525s^2 + 0,09366s + 0,006335}{1109s^4 + 968,8s^3 + 154,6s^2 + 1,035s} \quad (15)$$

En este gráfico se verifica la cancelación de los dos polos mas dominantes y el polo al origen que se agrega por el PID.

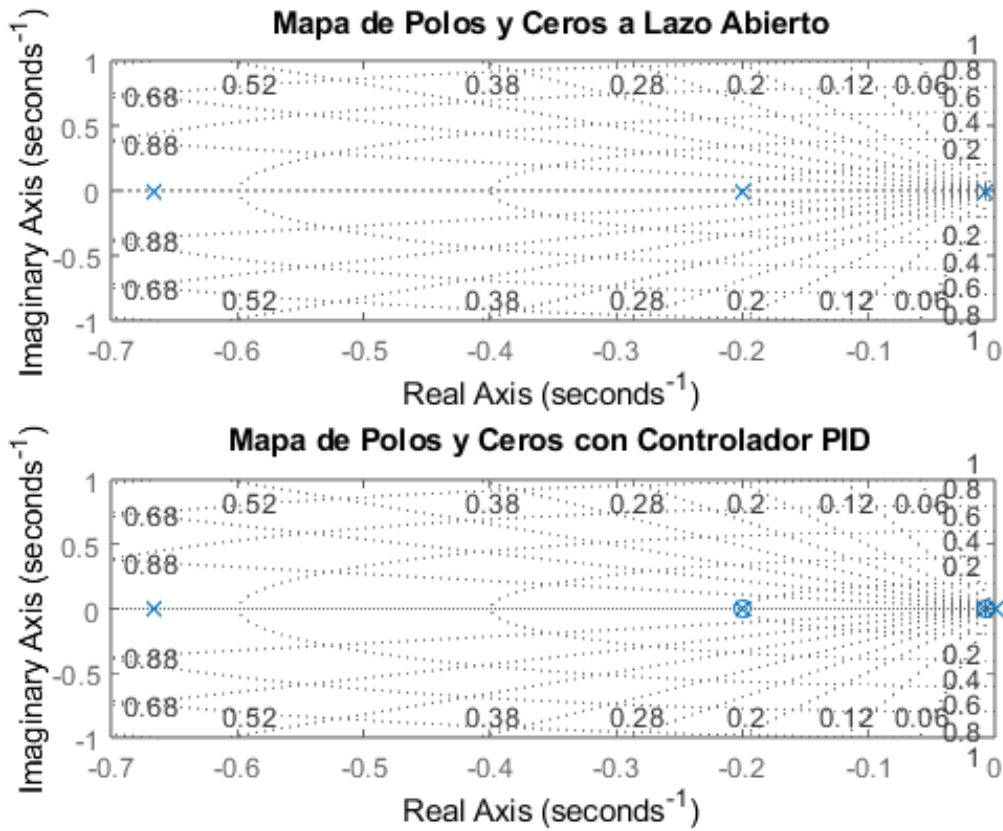


Figura 9: Polos y ceros, antes vs después de la compensación

El lugar de raíces confirma que el sistema termina siendo inestable para todo k_p , esto también descarta el compensador PI, por lo mencionado anteriormente. La única opción radica en un compensador PD, el cual lo desarrollamos a continuación.

Otra alternativa sería añadir un inversor, para que la ganancia total del sistema resulte positiva y poder utilizar el compensador PID para mejorar tanto la respuesta transitoria como el error en estado estable del sistema.

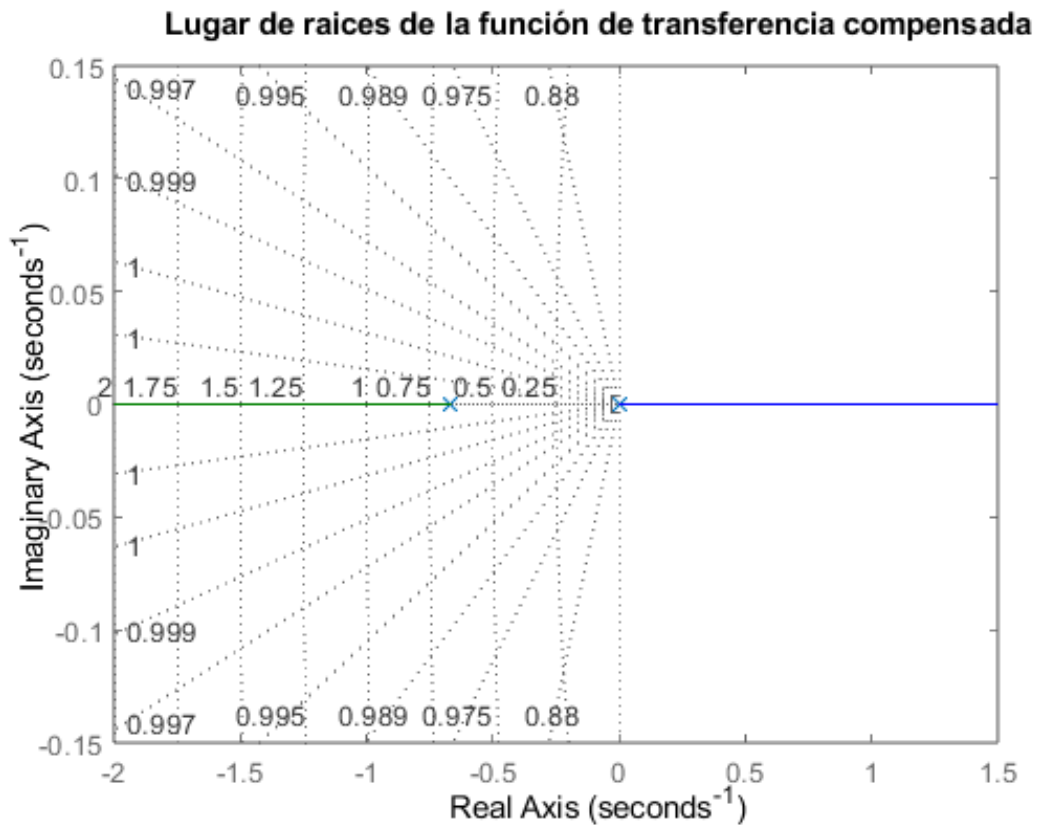


Figura 10: Lugar de raíces FdT compensado por PID

Para el compensador PD tenemos:

```

1  % La forma del PI es:  $PID = k_p T_d(s+1/T_d)$ 
2
3  % Teniendo en cuenta que el polo dominante es  $s=-0.007$  tenemos:
4  Td=1/0.007
5
6  % Armamos el controlador PI utilizando los valores de  $T_i$ 
7  PD = Td*(s+1/Td)
8
9  % Armamos la funcion de transferencia a lazo abierto compensado (con
10 PD)
11 FdT_Global_PD = PD*GH
12
13 % Verificamos la cancelacion de polos
14 figure;
15 subplot(2,1,1); % Primer subplot para la funcion a
16 pzmap(GH); % Muestra el mapa de polos y ceros de la funcion a
17 title('Mapa de Polos y Ceros a lazo abierto "GH"');
18 xlabel('real');
19 ylabel('imaginaria');
20 grid on;
21
22 subplot(2,1,2); % Segundo subplot para la funcion con PI
23 pzmap(FdT_Global_PD); % Muestra el mapa de polos y ceros de la
24 title('Mapa de Polos y Ceros con Controlador PD');
25 xlabel('real');

```

```

25 ylabel('imaginaria ');
26 grid on;
27
28 % Simplificamos la expresion para obtener una forma mas compacta
29 FdT_Global_PD = minreal(FdT_Global_PD) % Simplificacion de la fdt
30
31 figure;
32 rlocus(FdT_Global_PD);
33 grid on;
34 title('Lugar de raices de la funcion de transferencia compensada');
35
36 %Estable para kp<11
37 rlocfind(FdT_Global_PD);
38 grid on;

```

La función de transferencia del controlador PD resulta:

$$PD(s) = 142,9s + 1 \quad (16)$$

La función a lazo cerrado global compensada es:

$$FdT_{GlobalComp} = \frac{-0,0120}{s^2 + 0,8667s + 0,1333} \quad (17)$$

Verificamos la cancelación de polos dominantes y buscamos un valor de kp que verifique los requerimientos.

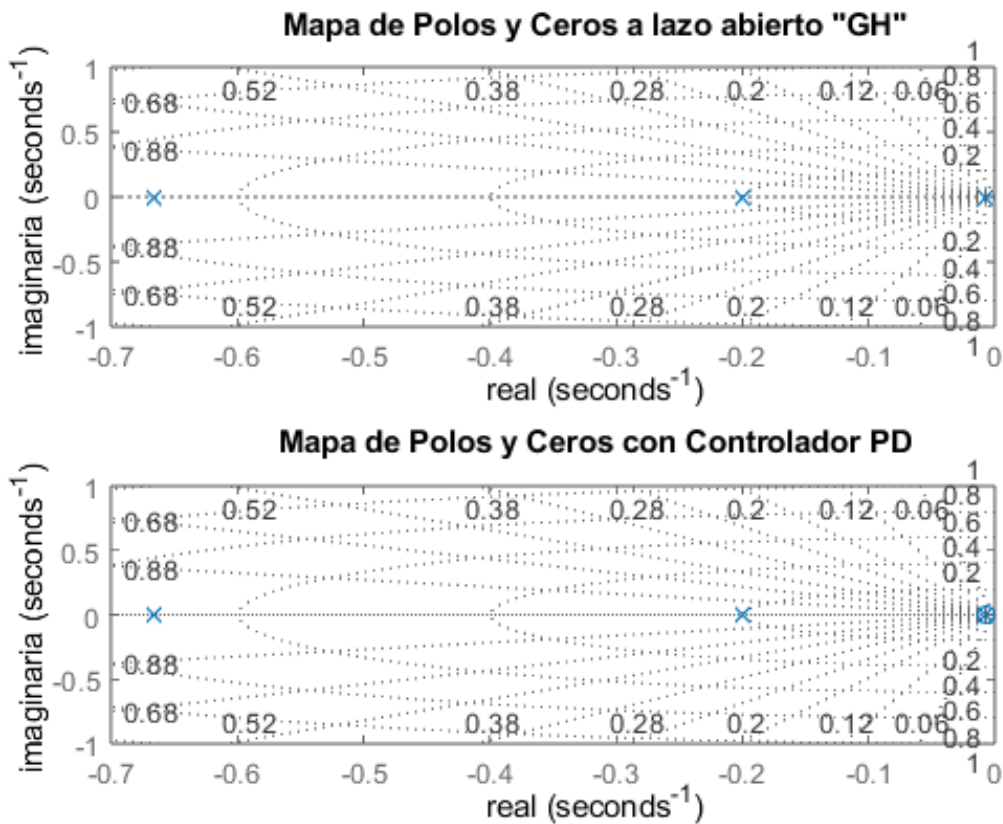


Figura 11: Cancelación de polos dominantes PD

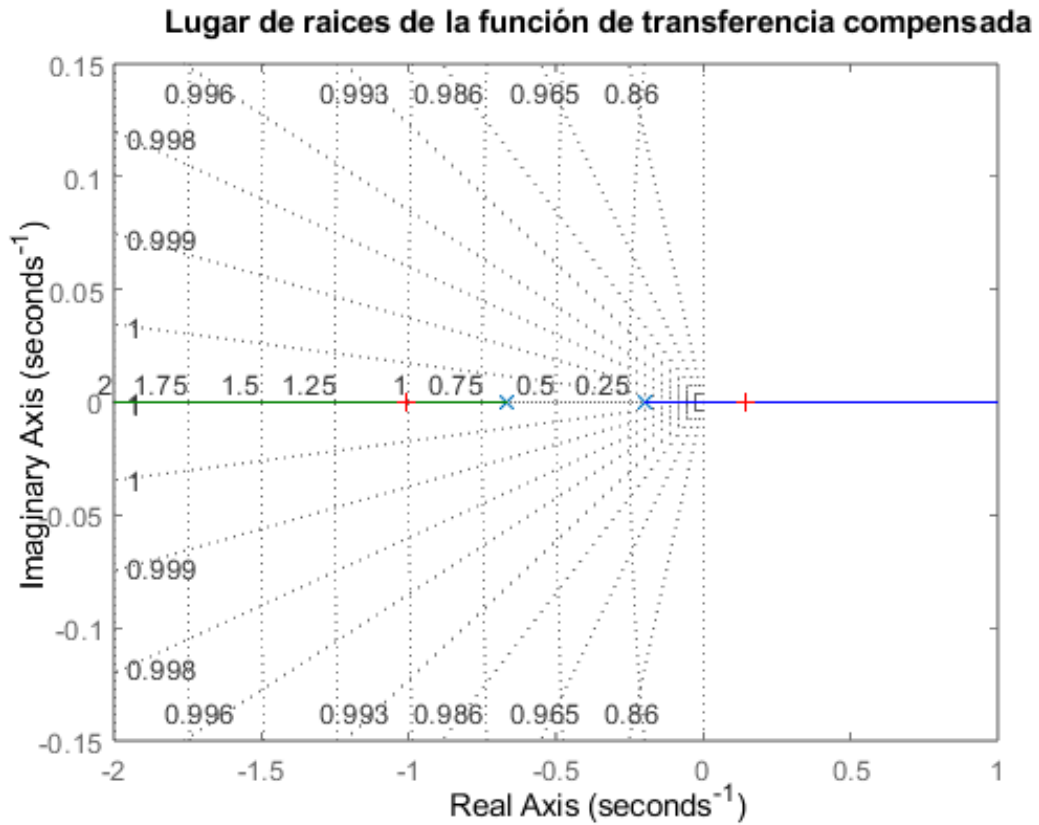


Figura 12: Lugar de raíces compensador PD

Analizando el lugar de raíces podemos concluir que el sistema es estable para todo $k_p < 11$, a menor el valor de este, mayor velocidad de respuesta se obtiene. Un valor adecuado de ganancia proporcional es $k_p = 1$ ya que cumple con el tiempo de establecimiento que se tiene como requerimiento

5. Simulación

En esta sección, nos interesa comparar la respuesta temporal antes y después de compensar, además de calcular el error de estado estable del sistema.

```
1 % El valor que satisface los requerimientos es  $k_p = 1$ 
2 figure;
3
4 kp=1;
5 step(feedback(G,H), feedback(kp*G*PD,H));
6 grid on;
7
8 title('Comparativa respuesta temporal');
9 legend('no compensado', 'compensado')
```

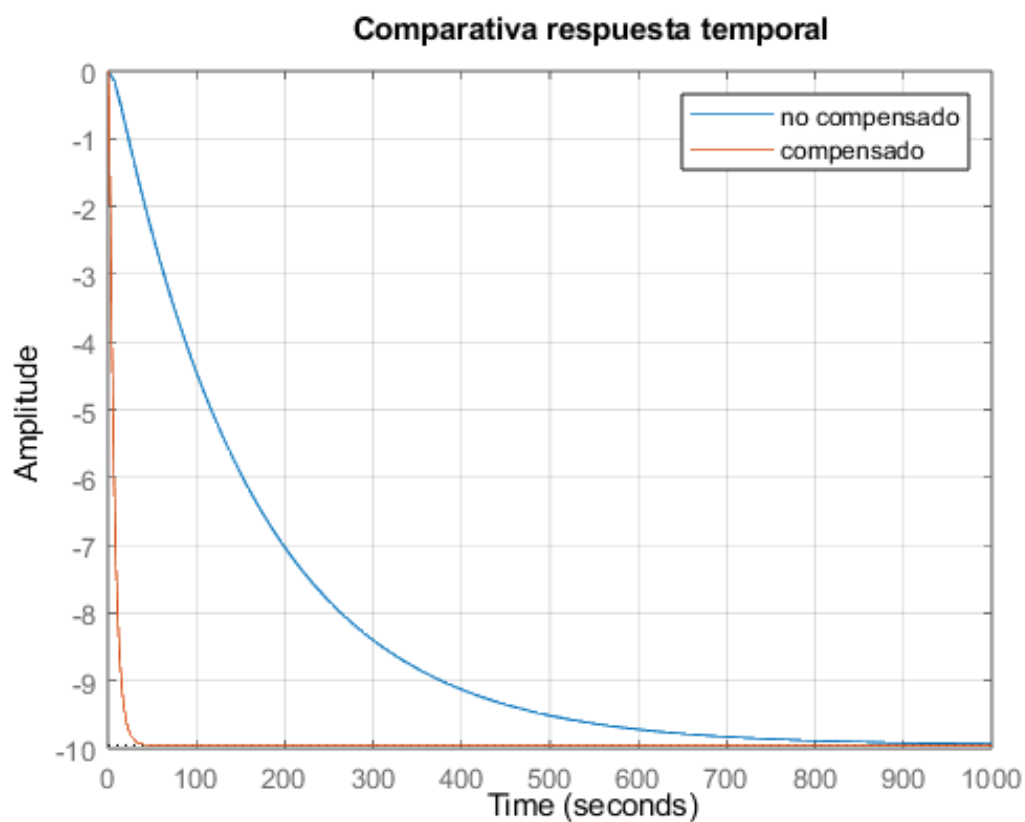


Figura 13: Respuesta temporal comparativa

Se puede ver una mejora significativa en el tiempo de establecimiento. Respecto al error en estado estable, podemos realizar el siguiente análisis. Sabemos que el sistema no tiene polos en el origen (Tipo 0), por lo que esperamos un error de estado estable constante frente a la respuesta de un escalón, luego:

```
1 %Análisis del error en estado estable, la fdt global resulta un
2 %sistema de
3 %tipo 0, por lo tanto se espera que el error en estado estable sea
4 %constante.
5
6 R=1; %amplitud escalon
7 kp=evalfr(G*H*PD,0);
8
9 disp('Error de estado estable resulta: ');
```


$$ess = R / (1 + k_p)$$

El error en estado estable es:

$$ess = 1,099 \quad (18)$$

Esta especificación no se puede mejorar, ya que aumentar el tipo del sistema lo vuelve inestable como vimos anteriormente con el compensador PID (Figura 10).

6. Conclusiones

Se han modelado todas las partes del sistema mediante diversos métodos para obtener la función de transferencia global, el sistema resulto ser marginalmente estable con una respuesta transitoria insatisfactoria. Mediante la utilización de un compensador proporcional-derivativo se mejora la respuesta transitoria en un 1400 % aproximadamente sin afectar al sobrepasamiento y ni al error de estado estable.

7. Bibliografía

Bibliografía de referencia:

- Presentaciones - Ing. Pedroni
- Sistemas de control automático - Benjamín.C.Kuo

Nota: Todos los archivos utilizados en el informe (matlab/simulink, documentación, imágenes, datos, etc) se encuentran en el siguiente link de repositorio:

<https://github.com/marian1911/Trabajo-Integrador-SCI.git>